

Component-and-connect patterns:

- Broker
- Model-view Controller
- **Pipe and Filter**
- Client-Server
- Peer-to-Peer
- Service-Oriented-Architecture
- Publish-Subscribe
- Shared-Data

Pipe and Filter Pattern

- **Context:** Many systems are required to transform streams of discrete data items, from input to output.
 - Many types of transformations occur repeatedly in practice, and so it is desirable to create these as independent, reusable parts.
- **Problem:** Such systems need to be divided into reusable, loosely coupled components with simple, generic interaction mechanisms.
 - In this way they can be flexibly combined with each other.
 - The components, being generic and loosely coupled, are easily reused.
 - The components, being independent, can execute in parallel.
- **Solution:** The pattern of interaction in the pipe-and-filter pattern is characterized by successive transformations of streams of data.
 - Data arrives at a filter's input port(s), is transformed, and then is passed via its output port(s) through a pipe to the next filter.
 - A single filter can consume data from, or produce data to, one or more ports.

Pipe and Filter Example

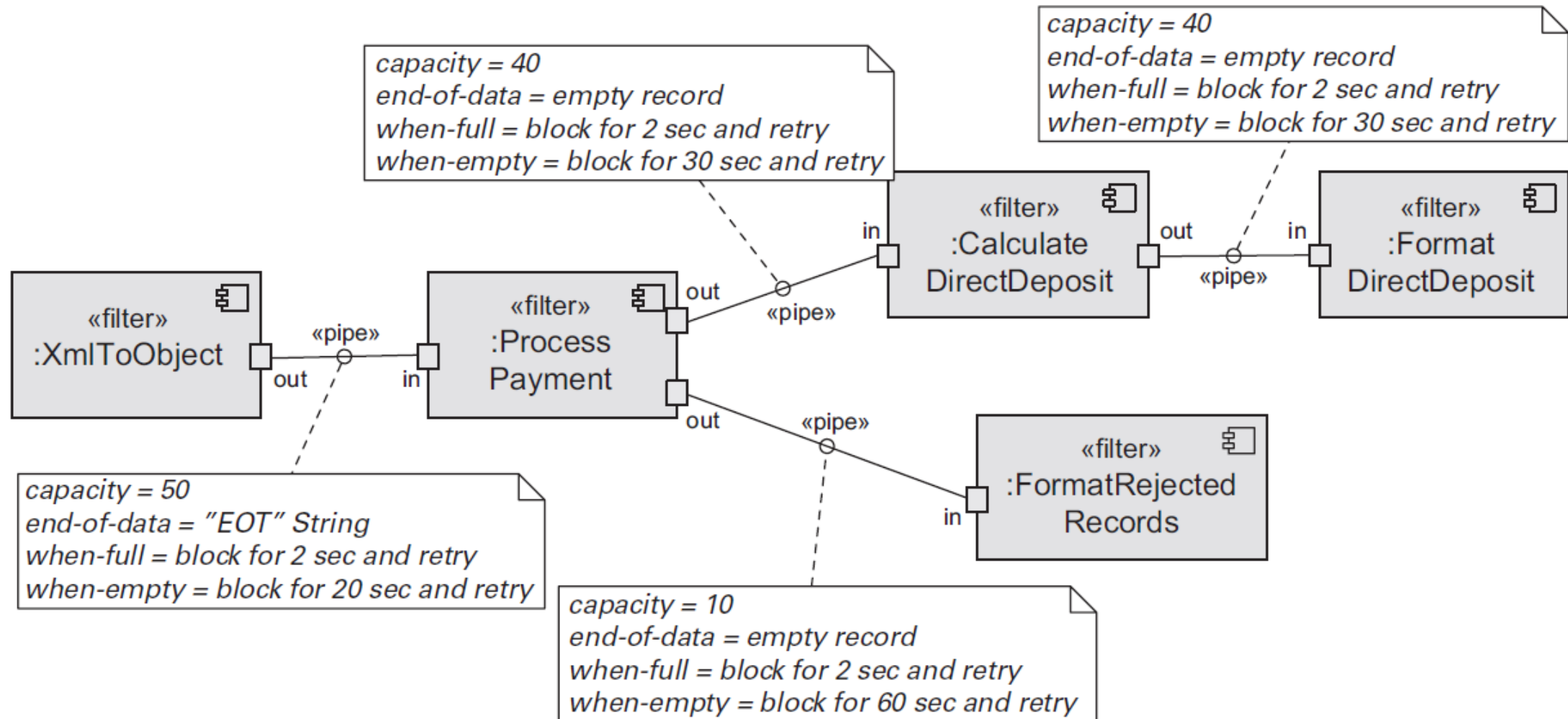


FIGURE 13.8 A UML diagram of a pipe-and-filter-based system

Pipe and Filter Solution

- Overview: Data is transformed from a system's external inputs to its external outputs through a series of transformations performed by its filters connected by pipes.
- **Elements:**
 - *Filter*, which is a **component** that **transforms data** read on its **input** port(s) to data written on its **output** port(s).
 - *Pipe*, which is a **connector** that **conveys data from a filter's output port(s) to another filter's input port(s)**.
 - A **pipe** has a **single source** for its **input** and a **single target** for its **output**.
 - A **pipe** **preserves the sequence of data items**, and it does **not alter the data passing through**.
- **Relations:** The *attachment* relation **associates the output of filters with the input of pipes** and vice versa.

Pipe and Filter Solution

- **Constraints:**

- Pipes connect filter output ports to filter input ports.
- Connected filters must agree on the type of data being passed along the connecting pipe.

- **Weaknesses:**

- Having large numbers of independent filters can add substantial amounts of computational overhead.
 - because each filter runs as its own thread or process
- Pipe-and-filter systems may not be appropriate for long-running computations without the addition of some form of checkpoint/restore functionality, as the failure of any filter (or pipe) can cause the entire pipeline to fail.

Finer Points for Pipe and Filter

- Pipes buffer data during communication. Because of this property, filters can execute asynchronously and concurrently.
- Data transformation systems are typically structured as pipes and filters,
 - with each filter responsible for one part of the overall transformation of the input data.
 - The independent processing at each step supports reuse, parallelization, and simplified reasoning about overall behavior.
 - Often such systems constitute the front end of signal-processing applications.
 - These systems receive sensor data at a set of initial filters; each of these filters compresses the data and performs initial processing (such as smoothing).
 - Downstream filters reduce the data further and do synthesis across data derived from different sensors.
 - The final filter typically passes its data to an application or example providing input to modeling or visualization tools.

Usages of Pipe and Filter

- Other systems that use pipe-and-filter include:
 - those built using UNIX pipes,
 - the request processing architecture of the Apache web server,
 - the map-reduce pattern,
 - Yahoo! Pipes for processing RSS feeds,
 - many workflow engines,
 - many scientific computation systems that have to process and analyze large streams of captured data